HKBU Campus Assistant

COMP7940 Cloud Computing Project

A professional academic project presentation for a 6-minute group showcase. Presented by Li Xiang, CHEN Yuqiao, and ZHANG Xinyi.

Part 1

Project Overview & LLM Architecture

Li Xiang · ~2 min

Part 2

DevOps Pipeline & Containerised Deployment

CHEN Yuqiao · ~2 min

Part 3

System Integration, Monitoring & Cost Optimisation

ZHANG Xinyi · ~2 min

PART 1 · LI XIANG

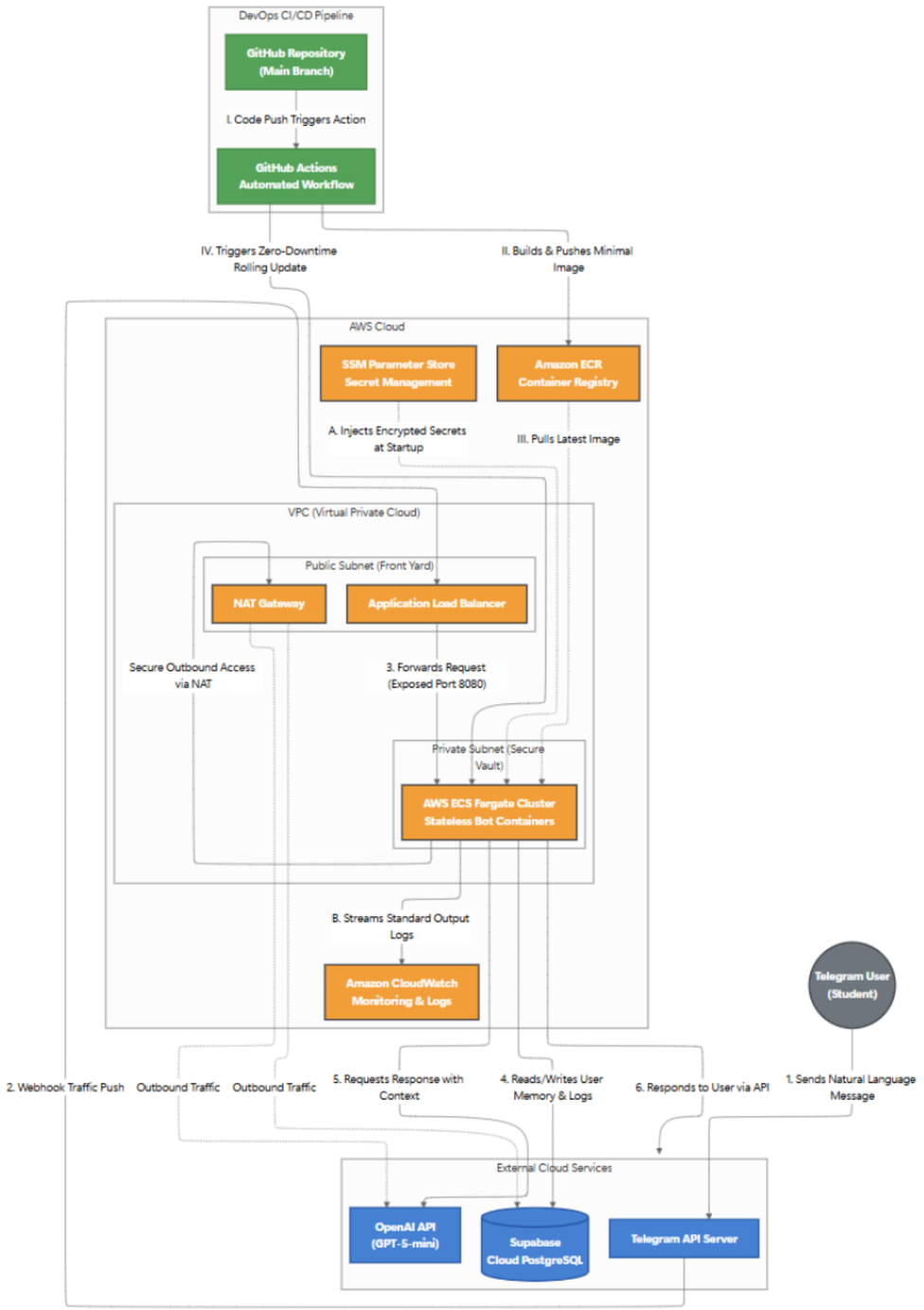
Slide 1: Project Background & Vision

Core Positioning

HKBU Campus Assistant -- cloud-native Telegram chatbot fully deployed on AWS with (NLP) to provide course Q&A and personalised recommendations,

Core Value

Use Cloud Agility: Utilizing AWS infrastructure provides a highly resilient and stable foundation. Our serverless compute approach dynamically adjusts resources, offering a modern, budget-friendly cloud practice.



High-level Architecture Diagram

PART 1 · LI XIANG

Slide 2: Core Features & LLM Integration

Technology Stack

Backend logic built on **Python**, providing a robust and extensible foundation for all campus assistant interactions.

AI Empowerment

Integrated with the **GPT-5-mini API**, with purpose-designed **Prompt templates** tailored specifically for campus scenarios.

Key Highlight

Goes beyond simple code stacking — we have constructed an **interactive framework with domain-specific knowledge**, enabling contextually aware and academically relevant responses.

PART 1 · LI XIANG

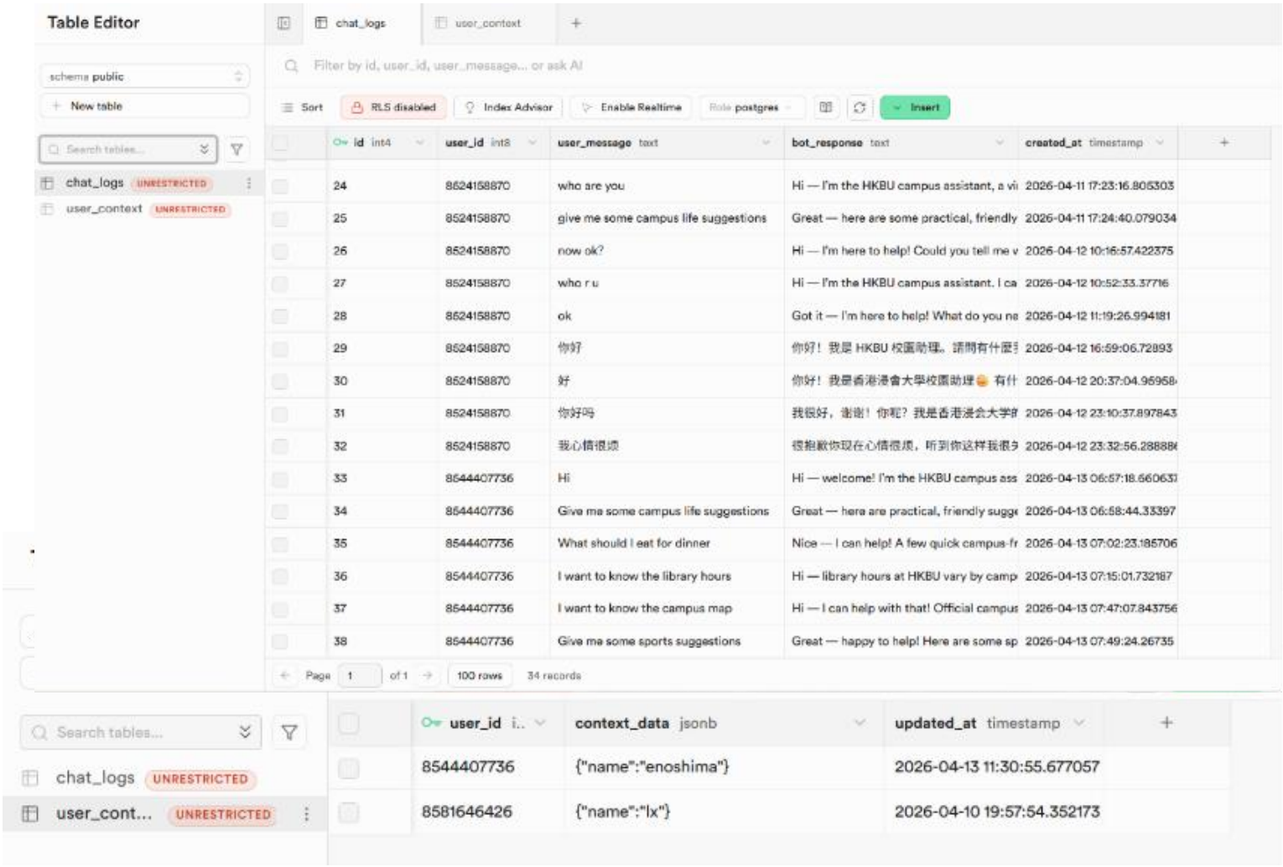
Slide 3: Cloud Database & "Context Memory" Persistence

Pain Point Solved

Traditional bots lack conversational continuity. We implemented **context-aware memory** via the `remember` command, enabling coherent multi-turn dialogue.

Data Architecture

Utilises **Supabase (Cloud PostgreSQL)** to store user logs and interaction history, guaranteeing cloud-side data persistence across sessions.



Supabase Database Persistence

PART 2 · CHEN YUQIAO

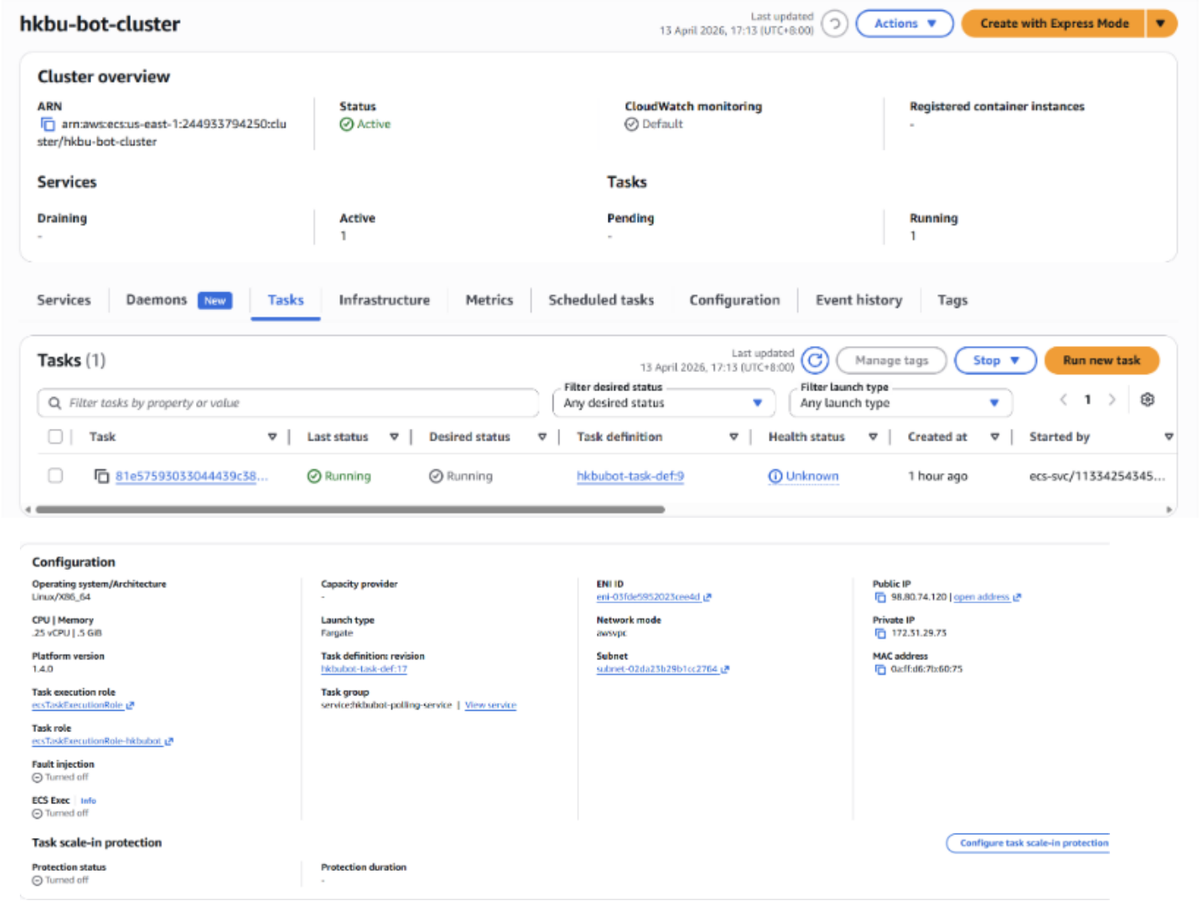
Slide 4: Containerisation & Serverless Computing

Technical Highlight — Multi-stage Docker Build

- Separates the **compilation** and **execution** environments
- Replaces source-code build dependencies with **pre-compiled binaries**
- Dramatically improves **container cold-start efficiency**

Cloud Deployment — AWS ECS + Fargate

- Deployed on **AWS ECS** with the **Fargate serverless** compute engine
- Achieves a fully **Stateless** backend architecture
- Dynamically adjusts compute resources based on utilisation

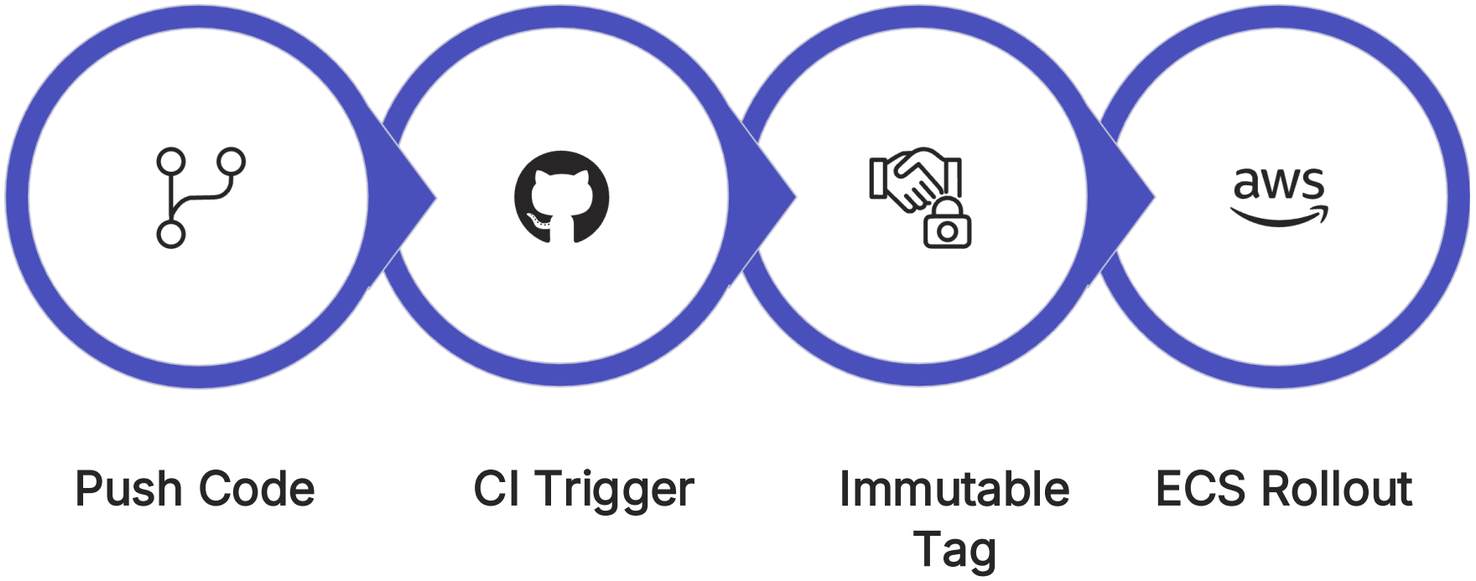


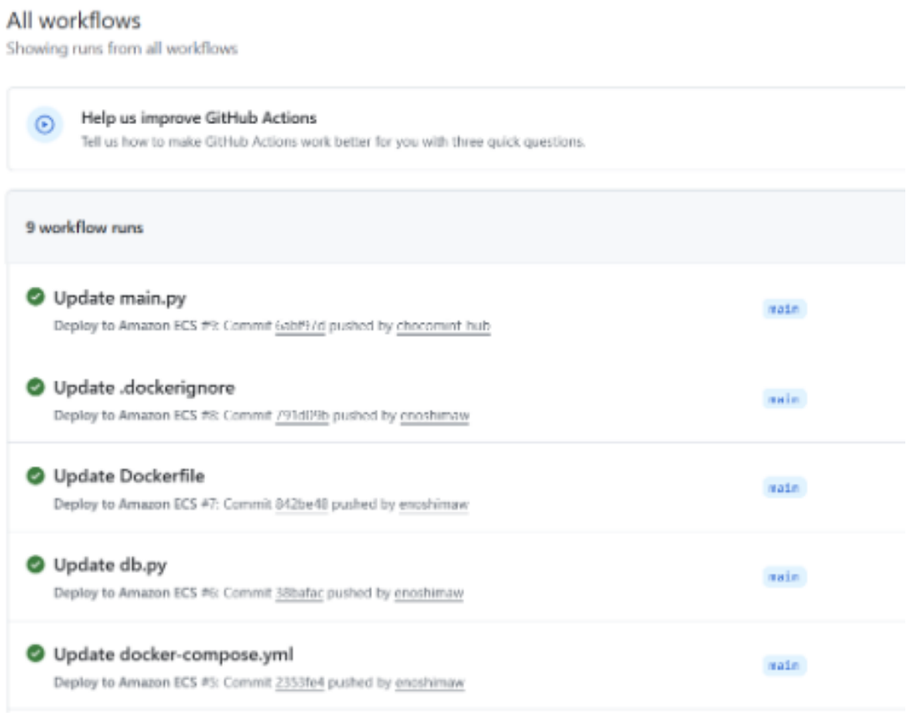
AWS ECS Fargate Cluster & Tasks

PART 2 · CHEN YUQIAO

Slide 5: Zero-Downtime Automated Deployment (CI/CD Pipeline)

Pain Point Solved: Eliminates the error-prone nature of manual, human-driven deployments.





The **ECS Rolling Update** strategy ensures that old instances are only destroyed *after* new containers pass their health checks, achieving seamless, zero-downtime migration. A Git commit hash serves as an immutable image version tag throughout the pipeline.

GitHub Actions Automated Workflow

PART 2 · CHEN YUQIAO

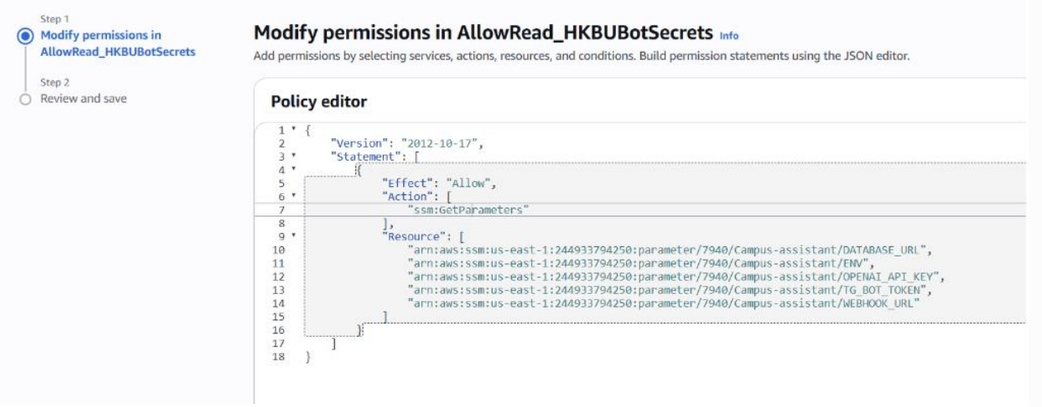
Slide 6: Defence-in-Depth & Cloud Security

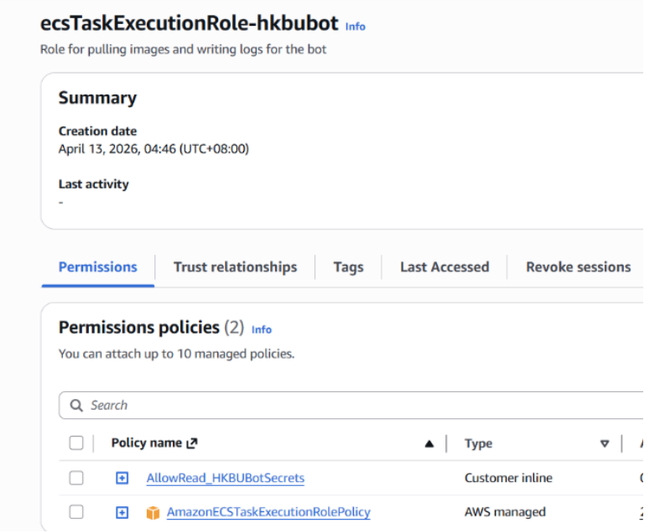
Configuration Isolation

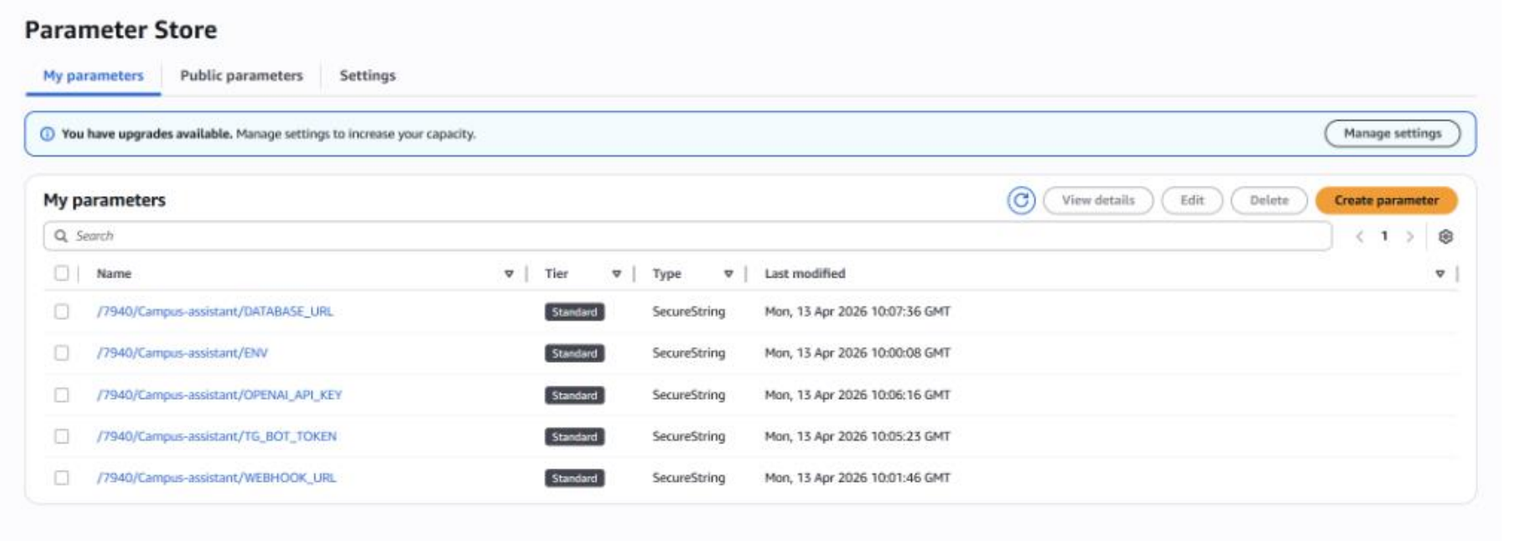
- Practises **12-Factor App** principles — zero hardcoded API Keys
- Encrypted environment variables dynamically injected via **AWS SSM Parameter Store**
- Equipped with a **Fail-fast** mechanism — missing variables trigger immediate termination

Network & Permissions

- Migrated from long-polling to an **event-driven (Webhook)** architecture
- Strict **Least Privilege Principle** enforced (e.g., `ecsTaskExecutionRole`)
- Security Groups configured to **block all active inbound traffic**, eliminating port-scanning threats







AWS SSM Parameter Store / IAM Roles

PART 3 · ZHANG XINYI

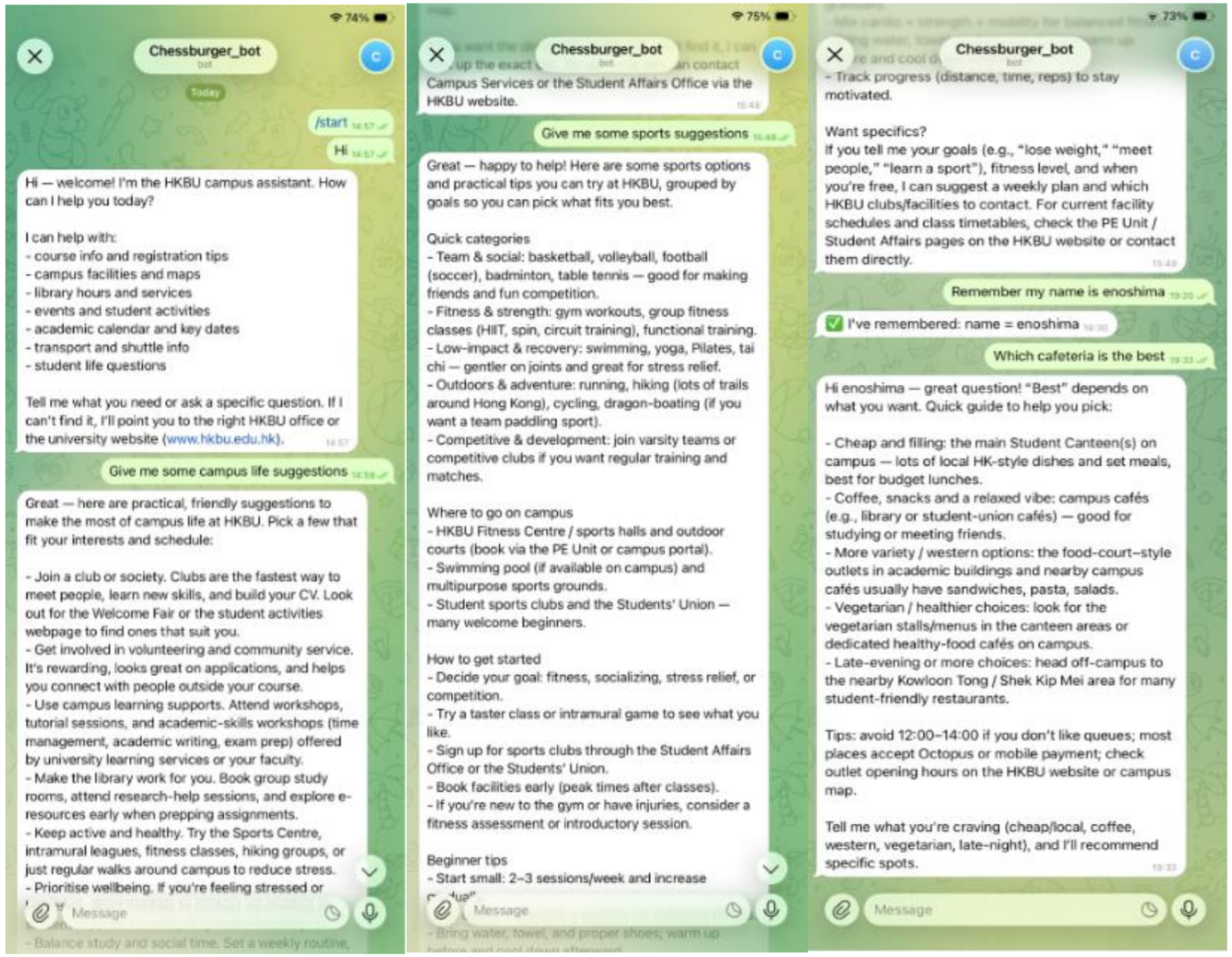
Slide 7: Frontend Integration & User Interaction

Platform Integration

Completed **Telegram Bot registration**, Token configuration, and message routing

Interaction Showcase

Demonstrates real-environment performance across three key capabilities:



Natural Language Dialogue

Contextually coherent multi-turn conversations

Campus Recommendation Retrieval

Domain-specific knowledge responses

Personalised Memory

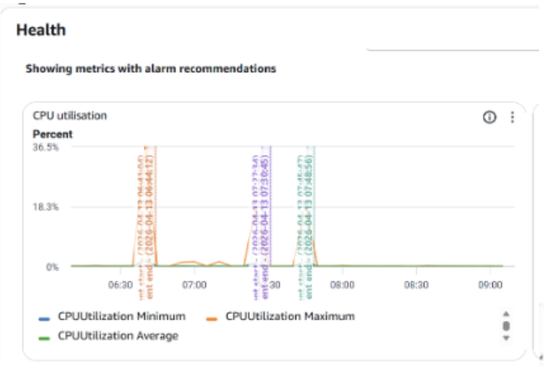
Live `remember` function responses

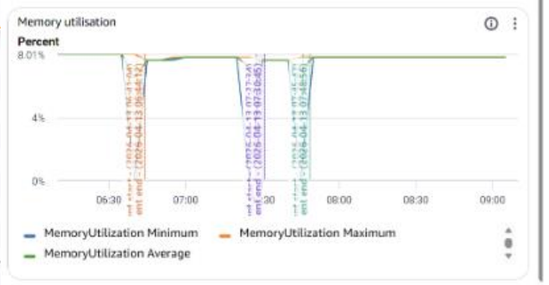
Slide 8 & 9: Observability & Cost Control

System Observability

Integrated **AWS CloudWatch** for comprehensive monitoring:

- Logs all **requests and responses** for real-time debugging and request analysis
- Tracks **CPU and memory utilisation** to ensure system health and stability

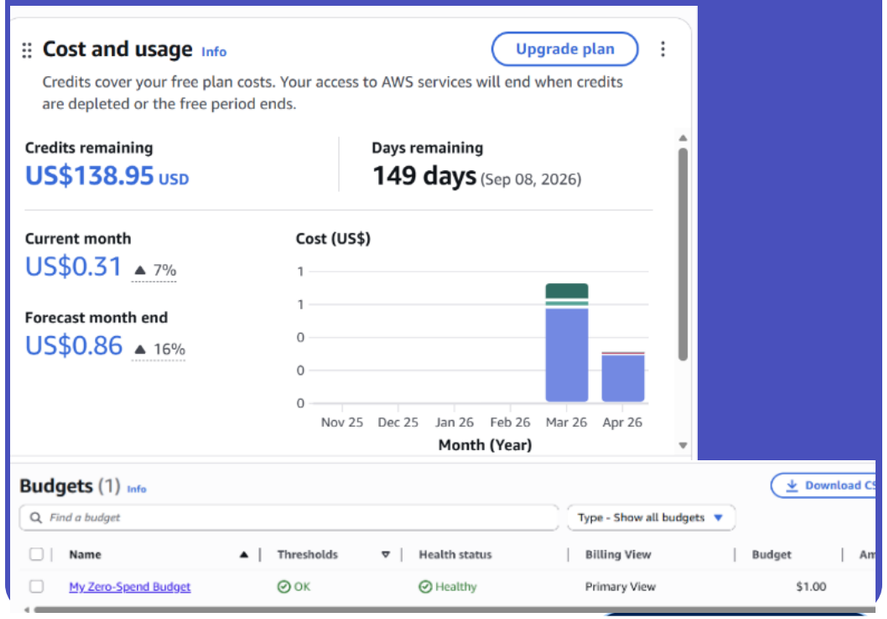




Cost Management

Current Billing: Costs remain under $1. The architecture is optimized for the AWS Free Tier to minimize resource usage without impacting performance.

Cost Control: We implemented a zero/low-cost AWS budget with automated email alerts to proactively prevent overspending.



Slide 10: Summary & Q&A

> A modern cloud-native microservice application that combines **AI intelligence**, a **Serverless high-efficiency architecture**, and **low-cost, high-security** design principles.

AI Intelligence

GPT-5-mini powered LLM with domain-specific Prompt templates and context-aware memory via Supabase.

Serverless Efficiency

AWS ECS Fargate with multi-stage Docker builds, CI/CD via GitHub Actions, and zero-downtime rolling updates.

Security & Cost

Least Privilege IAM, SSM Parameter Store, Webhook architecture, and total cloud spend under $1.00.

Thank You — Questions Welcome 🎓

Presented by **Li Xiang**, **CHEN Yuqiao**, and **ZHANG Xinyi** · COMP7940 Cloud Computing · HKBU